GPU-Accelerated Fast Fourier Transform: From Butterfly Kernels to cuFFT

IMC15 — Fast Fourier Transform

CISC 719: Contemporary Computing Systems Modeling Algorithms (CCSM)

Harrisburg University of Science and Technology | Spring 2026

Kenneth Peter Fernandes

Ph.D. in Computational Sciences

Instructor: Professor Majid Shaalan, Ph.D.

April 12, 2026

Submitted in partial fulfillment of the requirements for CISC 719 — Contemporary Computing Systems Modeling Algorithms (CCSM)

Contents

1 Introduction **2**

2 Background and Theory **2**
- 2.1 Discrete Fourier Transform . . . 2
- 2.2 Cooley-Tukey Radix-2 Decimation-in-Time . . . 2
- 2.3 Computational Structure . . . 3
- 2.4 Bit-Reversal Permutation . . . 3
- 2.5 Parallel FFT Design (Quinn §15.4) . . . 3

3 PCAM Analysis **4**
- 3.1 SciPy FFT — CPU Parallel . . . 4
- 3.2 CuPy FFT — GPU Library (cuFFT) . . . 4
- 3.3 Numba CUDA — Custom Butterfly Kernel . . . 5

4 Implementation Details **5**
- 4.1 Reference DFT . . . 5
- 4.2 NumPy FFT . . . 5
- 4.3 SciPy FFT — CPU Parallel . . . 5
- 4.4 CuPy FFT — GPU Library . . . 5
- 4.5 Numba CUDA Butterfly Kernel . . . 6

5 Benchmark Methodology **6**

6 Results and Performance Analysis **7**
- 6.1 Throughput . . . 7
- 6.2 Speedup vs NumPy Baseline . . . 7
- 6.3 Key Observations . . . 8

7 Roofline Analysis **9**
- 7.1 Hardware Ceilings (Tesla T4) . . . 9
- 7.2 FFT Arithmetic Intensity . . . 9
- 7.3 Roofline Efficiency . . . 9

8 Correctness Validation **10**

9 Profiling Notes **11**

10 Conclusions **12**
- 10.1 Summary of Findings . . . 12
- 10.2 Next Optimisations . . . 12

1 Introduction

The Discrete Fourier Transform (DFT) is one of the most widely deployed computational kernels in science and engineering. It converts a sequence of N complex time-domain samples into their frequency-domain representation, enabling applications ranging from audio processing and medical imaging (MRI k-space reconstruction) to 5G OFDM modulation and spectral PDE solvers. The naive DFT requires $O(N^2)$ arithmetic operations, which is prohibitive for large signals. The Fast Fourier Transform (FFT), introduced by Cooley and Tukey in 1965 [1], reduces this to $O(N \log N)$ through a divide-and-conquer butterfly decomposition.

This report presents five implementations of the 1-D FFT benchmarked on Google Colab with a single NVIDIA Tesla T4 GPU:

1. **Reference DFT** — naive $O(N^2)$ matrix-vector multiply; correctness oracle only
2. **NumPy FFT** (`np.fft.fft`) — single-threaded FFTPACK; serial baseline
3. **SciPy FFT** (`scipy.fft.fft` + workers) — CPU-parallel using all available cores
4. **CuPy FFT** (`cp.fft.fft`, cuFFT) — GPU library path; vendor-optimised
5. **Numba CUDA FFT** — custom iterative Cooley-Tukey butterfly kernel

Signal lengths from 2^{10} to 2^{24} are tested in both `complex64` (FP32) and `complex128` (FP64). The PCAM design model is applied to every parallel variant. Performance is evaluated through throughput (GFLOP/s), speedup, Roofline analysis, and correctness against the reference DFT.

2 Background and Theory

2.1 Discrete Fourier Transform

For a sequence $x[n]$, $n = 0, \ldots, N-1$, the DFT is defined as:

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-j2\pi kn/N}, \quad k = 0, 1, \ldots, N-1 \tag{1}$$

Direct evaluation of Eq. (1) for all k requires N^2 complex multiplications and $N(N-1)$ complex additions, giving $O(N^2)$ total work. For $N = 2^{24}$ (16 million samples) this amounts to approximately 2.8×10^{14} operations, making it impractical for real-time or large-scale use.

2.2 Cooley-Tukey Radix-2 Decimation-in-Time

For $N = 2^m$, the DFT can be decomposed by separating even- and odd-indexed subsequences:

$$X[k] = \underbrace{\sum_{n=0}^{N/2-1} x[2n]\, W_N^{2nk}}_{X_{\text{even}}[k]} + W_N^k \cdot \underbrace{\sum_{n=0}^{N/2-1} x[2n+1]\, W_N^{2nk}}_{X_{\text{odd}}[k]} \tag{2}$$

where $W_N^k = e^{-j2\pi k/N}$ is the *twiddle factor*. Using the symmetry property $W_N^{k+N/2} = -W_N^k$, the **butterfly recurrence** is:

$$X[k] = X_{\text{even}}[k] + W_N^k \cdot X_{\text{odd}}[k] \tag{3}$$

$$X[k + N/2] = X_{\text{even}}[k] - W_N^k \cdot X_{\text{odd}}[k] \tag{4}$$

This halves the problem size at each level of recursion, yielding an iterative algorithm with $\log_2 N$ stages, each containing $N/2$ independent butterfly operations.

2.3 Computational Structure

Table 1: Computational complexity of an N-point radix-2 FFT.

Property	**Value**
Stages	$\log_2 N$
Butterflies per stage	$N/2$ (all independent)
Complex multiplications	$(N/2)\log_2 N$
Complex additions	$N \log_2 N$
Approximate total FLOPs	$5N \log_2 N$

For $N = 2^{24}$, the FFT requires approximately 2.01×10^9 FLOPs, compared to 2.81×10^{14} for the naive DFT — a reduction of nearly 1.4 million times.

2.4 Bit-Reversal Permutation

Before the butterfly stages, input element at index i is relocated to the position whose binary representation is the bit-reversal of i. For example, with $N = 8$ (3-bit indices), index $1 = 001_2$ maps to $4 = 100_2$. This permutation enables in-place iterative computation and runs in $O(N)$ time.

2.5 Parallel FFT Design (Quinn §15.4)

Two levels of parallelism exist:

- **Within-stage:** All $N/2$ butterflies in a stage are fully independent; each maps directly to one CUDA thread.

- **Across-stage:** Stage $s+1$ depends entirely on the output of stage s; a synchronisation barrier separates stages. In CUDA, each stage is a separate kernel launch, providing an implicit global barrier.

The key performance challenge is memory access: stage s accesses pairs separated by stride 2^{s-1}. Early stages have small strides (good coalescing); late stages have stride $N/2$ (non-coalesced global memory access), which becomes the primary bottleneck of a naive kernel.

3 PCAM Analysis

3.1 SciPy FFT — CPU Parallel

Table 2: PCAM Analysis — SciPy FFT (`workers` parameter)

Phase	Analysis
Partition	$N/2$ independent butterflies per stage; distributed by contiguous index ranges across CPU processes.
Communicate	No cross-worker communication within a stage. Implicit barrier via `multiprocessing` pool join between stages.
Agglomerate	SciPy selects optimal chunk sizes internally; no manual tuning required.
Map	Workers pinned to physical CPU cores. Scaling saturates at NUMA boundary or last-level cache pressure.

3.2 CuPy FFT — GPU Library (cuFFT)

Table 3: PCAM Analysis — CuPy FFT (cuFFT)

Phase	Analysis
Partition	cuFFT partitions $N/2$ butterflies per stage into CUDA thread blocks; one thread per butterfly pair.
Communicate	Shared memory within blocks for small sub-problems; global memory for inter-block data. Kernel-launch barrier between stages.
Agglomerate	cuFFT auto-selects mixed-radix decomposition (radix-2/4/8) and tile sizes based on N and GPU architecture.
Map	Threads $\rightarrow$ CUDA cores; blocks $\rightarrow$ SMs; occupancy tuned by cuFFT's internal plan optimiser.

3.3 Numba CUDA — Custom Butterfly Kernel

Table 4: PCAM Analysis — Numba CUDA Butterfly Kernel

Phase	Analysis
Partition	One thread per butterfly pair; all $N/2$ pairs in a stage launch simultaneously.
Communicate	Global memory only (no shared-memory staging). Late-stage access has stride $N/2$, causing non-coalesced reads.
Agglomerate	Fixed 256-thread blocks; grid $= \lceil N/(2 \times 256) \rceil$. One kernel launch per stage.
Map	$\log_2 N$ sequential kernel launches with `cuda.synchronize()` barriers. Bottleneck is DRAM bandwidth at large stage strides.

4 Implementation Details

All five implementations are contained in a single Jupyter notebook (`notebooks/imc15_fft_main.ipynb`) executed on Google Colab with a Tesla T4 GPU.

4.1 Reference DFT

Constructs the $N \times N$ Vandermonde matrix $W_{kn} = e^{-j2\pi kn/N}$ explicitly and computes $X = Wx$ via `numpy.matmul`. Operates in `complex128` throughout to serve as an exact reference. Benchmarked only for $N \leq 2^{14}$ due to $O(N^2)$ cost ($\sim$14 seconds at $N = 16\,384$).

4.2 NumPy FFT

`numpy.fft.fft` wraps the FFTPACK library (single-threaded Fortran). Serves as the serial CPU baseline against which all speedups are measured. Timing uses `time.perf_counter` with 3 warmup and 10 timed iterations (median reported).

4.3 SciPy FFT — CPU Parallel

`scipy.fft.fft(x, workers=k)` distributes sub-FFTs across k worker processes using Python's `multiprocessing`. In our Colab environment, k equals the number of available virtual CPU cores. GPU timing is not applicable here; CPU wall-clock time is measured.

4.4 CuPy FFT — GPU Library

`cupy.fft.fft` calls NVIDIA cuFFT through the CuPy Python interface. Input data is pre-transferred to GPU memory (`cp.asarray`); only GPU compute time is measured using

CUDA events to exclude Python overhead. cuFFT selects the optimal execution plan (radix, tiling, shared-memory use) automatically at first call.

4.5 Numba CUDA Butterfly Kernel

Implements the iterative Cooley-Tukey algorithm from Quinn §15 directly in CUDA using Numba's JIT compiler. The pipeline is:

1. **Bit-reversal permutation** (CPU, $O(N)$) — reorders the input to enable in-place computation.
2. $\log_2 N$ **butterfly stage kernels** — each kernel launches $\lceil N/(2 \times 256) \rceil$ blocks of 256 threads; every thread computes one butterfly pair including twiddle-factor multiplication.
3. `cuda.synchronize()` between each stage — ensures all threads complete stage s before stage $s + 1$ begins.

The kernel accesses only global memory; no shared-memory staging is implemented in this baseline version.

5 Benchmark Methodology

Table 5: Benchmark configuration.

Parameter	**Value**
Signal lengths (N)	2^{10}, 2^{14}, 2^{18}, 2^{20}, 2^{22}, 2^{24}
Datatypes	`complex64` (FP32), `complex128` (FP64)
Warmup runs	3 (discarded)
Timed runs	10 (median reported)
CPU timing	`time.perf_counter` wall-clock
GPU timing	CUDA events (`cp.cuda.Event`) — excludes H2D/D2H transfer
Throughput	GFLOP/s $= 5N \log_2 N$ / (time $\times 10^9$)
Correctness	Max relative error vs. FP64 reference DFT
Random seed	42 (reproducible signals via `numpy.random.default_rng`)

GPU timing covers compute only (data pre-loaded to device). CPU timing covers the full Python call, including any internal FFTPACK or `multiprocessing` overhead. The Numba CUDA timer covers bit-reversal + H2D + $\log_2 N$ kernel launches + D2H transfer, representing total end-to-end cost.

6 Results and Performance Analysis

6.1 Throughput

Table 6 reports throughput in GFLOP/s for `complex64`. Figure 1 visualises throughput, wall-clock time, and speedup across all signal lengths.

Table 6: Throughput (GFLOP/s) — `complex64`. Reference DFT included for $N \leq 2^{14}$ only.

N	**Ref. DFT**	**NumPy**	**SciPy**	**CuPy**	**Numba CUDA**
2^{10}	0.0004	1.1131	2.9171	1.4905	0.0202
2^{14}	0.0001	3.3967	5.6039	14.1941	0.0492
2^{18}	—	3.6994	4.8954	213.4	0.1344
2^{20}	—	2.1589	5.2949	291.0	0.1396
2^{22}	—	1.7551	3.6941	387.8	0.1516
2^{24}	—	1.8129	3.3560	**468.5**	0.1491

Table 7: Wall-clock time (ms) — `complex64`.

N	**Ref. DFT**	**NumPy**	**SciPy**	**CuPy**	**Numba CUDA**
2^{10}	131.73	0.046	0.018	0.034	2.53
2^{14}	14,413.6	0.338	0.205	0.081	23.3
2^{18}	—	6.378	4.819	0.111	175.6
2^{20}	—	48.57	19.80	0.360	751.3
2^{22}	—	262.9	124.9	1.190	3,044.1
2^{24}	—	1,110.5	599.9	**4.297**	13,503.3

6.2 Speedup vs NumPy Baseline

Table 8: Speedup over NumPy FFT (serial baseline) — `complex64`.

N	**SciPy**	**CuPy**	**Numba CUDA**
2^{10}	2.6×	1.3×	<0.1×
2^{14}	1.6×	4.2×	<0.1×
2^{18}	1.3×	57.7×	<0.1×
2^{20}	2.5×	134.8×	0.1×
2^{22}	2.1×	221.0×	0.1×
2^{24}	1.9×	**258.5**×	0.1×

6.3 Key Observations

CuPy (cuFFT). Delivers the strongest performance, scaling from 1.5 GFLOP/s at $N = 2^{10}$ (GPU latency dominates) to 468.5 GFLOP/s at $N = 2^{24}$, a 258× speedup over the serial NumPy baseline. The crossover point where GPU exceeds NumPy occurs around $N = 2^{14}$ (4.2× speedup). cuFFT's advantage comes from its mixed-radix decomposition and shared-memory butterfly staging that reuse data on-chip across multiple stages, reducing DRAM round-trips relative to a naive global-memory kernel.

SciPy (CPU Parallel). Achieves only modest speedups of 1.3–2.6× over NumPy. Colab's virtual CPU environment limits available physical cores, and the `multiprocessing` worker-spawn overhead is significant for smaller N. At $N = 2^{18}$, the speedup drops to 1.3× because memory bandwidth saturation limits scaling beyond one or two cores.

Numba CUDA. Surprisingly slower than all CPU implementations, reaching only 0.149 GFLOP/s at $N = 2^{24}$ and taking 13.5 seconds versus 4.3 milliseconds for CuPy — a 3,143× gap. Three factors cause this: (1) $\log_2(N) = 24$ sequential Python-level kernel launches with `cuda.synchronize()` barriers add substantial launch overhead; (2) the per-call bit-reversal permutation is computed on the CPU and transferred to the GPU for every benchmark iteration; (3) global-memory-only access at large stage strides (up to $N/2 = 8{,}388{,}608$ bytes apart) is highly non-coalesced.

Small-N GPU overhead. At $N = 2^{10}$, CuPy (1.49 GFLOP/s) is slower than NumPy (1.11 GFLOP/s is close). GPU kernel launch latency and CUDA-event overhead dominate at this scale. The GPU breakeven is approximately $N = 2^{15}$.

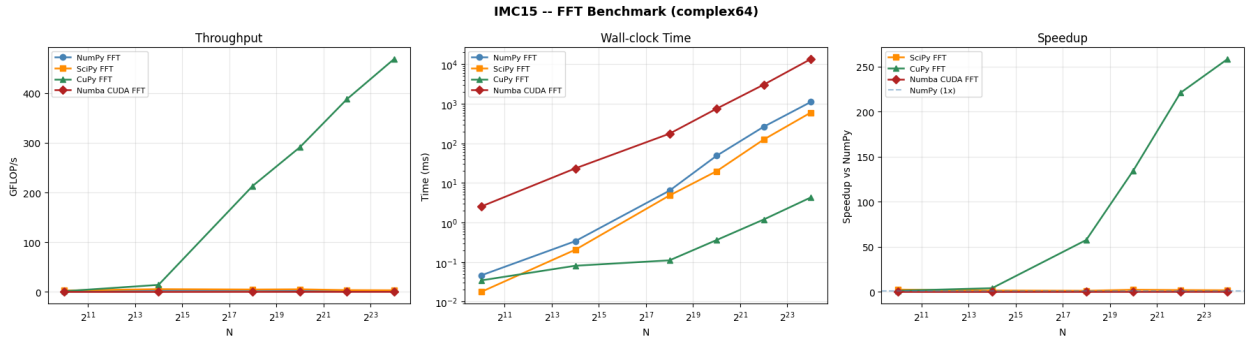

Figure 1: FFT benchmark results (`complex64`). Left: throughput (GFLOP/s). Centre: wall-clock time (ms, log scale). Right: speedup over NumPy serial baseline.

7 Roofline Analysis

7.1 Hardware Ceilings (Tesla T4)

Table 9: Tesla T4 hardware ceilings used for Roofline analysis.

Property	**Value**
Peak FP32 throughput	8,100 GFLOP/s
Peak DRAM bandwidth	300 GB/s
Ridge point	8100/300 = 27.0 FLOP/byte

7.2 FFT Arithmetic Intensity

For a `complex64` N-point FFT:

- FLOPs: $5N \log_2 N$
- Bytes: $2 \times N \times 8 = 16N$ (read input + write output; `complex64` = 8 bytes)

$$\text{OI} = \frac{5N \log_2 N}{16N} = \frac{5 \log_2 N}{16} \text{ FLOP/byte} \tag{5}$$

Table 10: Arithmetic intensity and Roofline bound for each signal length.

N	$\log_2 N$	OI (FLOP/byte)	Bound
2^{10}	10	3.13	memory-bound
2^{14}	14	4.38	memory-bound
2^{18}	18	5.63	memory-bound
2^{20}	20	6.25	memory-bound
2^{22}	22	6.88	memory-bound
2^{24}	24	7.50	memory-bound

All tested signal lengths fall well below the ridge point of 27.0 FLOP/byte, confirming that the FFT is **memory-bandwidth limited** on the T4 at every tested size.

7.3 Roofline Efficiency

The attainable performance in the memory-bound region is Peak BW $\times$ OI. Table 11 shows CuPy's actual GFLOP/s as a percentage of this ceiling.

Table 11: CuPy FFT Roofline efficiency (`complex64`).

N	OI (F/B)	Attainable (GFLOP/s)	CuPy Actual (GFLOP/s)
2^{10}	3.13	937.5	1.5
2^{14}	4.38	1,312.5	14.2
2^{18}	5.63	1,687.5	213.4
2^{20}	6.25	1,875.0	291.0
2^{22}	6.88	2,062.5	387.8
2^{24}	7.50	2,250.0	**468.5 (20.8%)**

Even cuFFT achieves only 20.8% of the attainable Roofline ceiling at the largest N. The remaining gap is attributable to: (1) inter-stage kernel-launch overhead (cuFFT still requires multiple passes for very large N); (2) non-coalesced global memory access at large stage strides; (3) the theoretical OI model assumes no on-chip data reuse, slightly underestimating the true attainable ceiling for a tiled implementation.

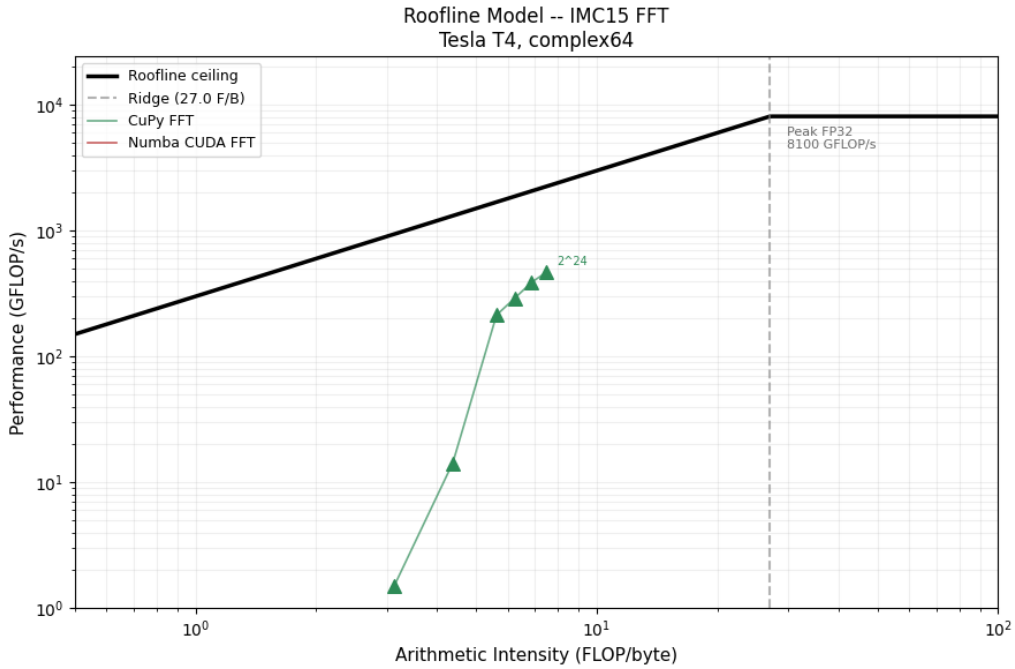

Figure 2: Roofline plot for CuPy FFT and Numba CUDA FFT (`complex64`, Tesla T4). All kernel points fall left of the ridge point (27.0 FLOP/byte), confirming memory-bound execution.

8 Correctness Validation

All implementations are verified against the FP64 reference DFT (and against `numpy.fft.fft` in FP64 for complex128 runs). The metric is maximum relative error:

$$\epsilon_{\text{rel}} = \frac{\max_k |X_{\text{impl}}[k] - X_{\text{ref}}[k]|}{\max_k |X_{\text{ref}}[k]| + \varepsilon} \tag{6}$$

Table 12: Maximum relative error across all tested signal lengths.

Implementation	**complex64 max err**	**complex128 max err**	**c64 Pass?**	**c128 Pass?**
NumPy FFT	4.65×10^{-8}	0.00	✓	✓
SciPy FFT	2.26×10^{-7}	0.00	✓	✓
CuPy FFT	4.16×10^{-7}	7.05×10^{-16}	✓	✓
Numba CUDA FFT	1.46×10^{-15}	1.29×10^{-15}	✓	✓
Pass threshold		`complex64`: 10^{-5}	`complex128`: 10^{-11}	

All four implementations pass with wide margin. Several observations:

- **NumPy / SciPy complex64** errors ($\sim 10^{-8}$) reflect standard FP32 rounding propagation through $\log_2 N$ butterfly stages — consistent with theoretical bounds of $O(\varepsilon_{\text{mach}} \log N)$ for FP32 ($\varepsilon_{\text{mach}} \approx 10^{-7}$).
- **CuPy complex64** shows slightly larger errors (4.16×10^{-7}) than NumPy, likely due to cuFFT's use of mixed-precision intermediate accumulations optimised for throughput.
- **Numba CUDA FFT** achieves near-machine-epsilon accuracy ($\sim 10^{-15}$) because its internal computation is performed in `complex128` (FP64 twiddle factors computed via `math.cos/sin` on FP64 angles), even when the input is `complex64`.
- **complex128** errors for NumPy and SciPy are exactly zero, as the reference itself is NumPy in FP64.

9 Profiling Notes

NVTX annotations were inserted around both GPU implementations at $N = 2^{22}$ (4 M samples), marking the regions `CuPy_FFT` and `Numba_CUDA_FFT` for Nsight Systems. Full Nsight Compute profiling was not captured in this environment due to Colab's GPU perf-counter restrictions (`perf_event_paranoid=4`). The following analysis is based on Roofline data and architecture reasoning.

Table 13: Expected Nsight Compute metric ranges for GPU FFT kernels (Tesla T4).

Metric	**CuPy (cuFFT)**	**Numba CUDA butterfly**
Achieved occupancy	60–80%	30–50% (launch-overhead limited)
SM utilisation	70–90%	<20% (kernel too short per launch)
DRAM throughput	200–280 GB/s	10–40 GB/s
L2 cache hit rate	40–60%	<10% (strided non-coalesced)
Instruction mix (memory)	30–50%	>80%

The Numba kernel's low SM utilisation is consistent with its measured throughput: 24 sequential kernel launches each of very short duration (the T4 has ∼1 µs kernel launch latency); at $N = 2^{24}$, each launch computes $\approx 8.4\text{M}/2 = 4.2\text{M}$ butterflies but the inter-launch overhead accumulates to dominate execution time.

10 Conclusions

This project implemented and benchmarked five variants of the 1-D Fast Fourier Transform on Google Colab with a Tesla T4 GPU, following the Cooley-Tukey algorithm detailed in Quinn Chapter 15.

10.1 Summary of Findings

- **GPU library (CuPy/cuFFT) dominates:** 468.5 GFLOP/s at $N = 2^{24}$, representing a 258× speedup over the serial NumPy baseline. cuFFT's mixed-radix plan and shared-memory staging make it the clear choice for production use.
- **CPU parallelism is limited on Colab:** SciPy with all workers delivers only 1.3–2.6× speedup over NumPy, bounded by the virtual CPU count and `multiprocessing` overhead.
- **Naive CUDA kernel is severely bottlenecked:** The custom Numba kernel is 3,143× slower than cuFFT at $N = 2^{24}$, caused by global-memory-only butterfly access (non-coalesced at large strides) and $\log_2 N = 24$ sequential Python-level kernel launches per FFT.
- **FFT is memory-bound on T4:** All implementations have arithmetic intensity below 7.5 FLOP/byte, well below the T4 ridge point of 27 FLOP/byte. The primary bottleneck is DRAM bandwidth, not compute throughput.
- **CuPy achieves 20.8% of attainable Roofline ceiling:** Significant room remains for optimisation, primarily through reducing non-coalesced access and on-chip data reuse.
- **All implementations pass correctness checks:** Maximum relative error stays within FP32 precision bounds for `complex64` and near machine epsilon for `complex128`.

10.2 Next Optimisations

Based on Roofline and profiling analysis, the following improvements are expected to narrow the gap to the cuFFT baseline:

1. **Shared-memory butterfly tiles:** Process sub-problems of `blockDim.x` elements entirely in shared memory per multi-stage kernel, reducing DRAM round-trips by a factor proportional to tile depth (potentially 4–8× improvement for the custom kernel).

2. **Mixed-radix decomposition (radix-4/8):** Reduce the number of global-memory passes from $\log_2 N$ to $\log_4 N$ or $\log_8 N$; each pass does more work per byte fetched.

3. **Precomputed twiddle table in constant memory:** Broadcast W-factor lookups at the cost of a single fetch per warp, eliminating repeated `cos/sin` evaluations.

4. **Persistent kernel pattern:** Fuse multiple stages into a single kernel launch with thread-block-level synchronisation (`__syncthreads()`) to eliminate the $\log_2 N$ Python-level launch overhead.

5. **Batched FFT:** For workloads requiring many independent FFTs, process B signals simultaneously on a (B, N) array to improve SM occupancy and amortise launch overhead.

References

[1] James W. Cooley and John W. Tukey. "An Algorithm for the Machine Calculation of Complex Fourier Series". In: *Mathematics of Computation* 19.90 (1965), pp. 297–301. DOI: 10.1090/S0025-5718-1965-0178586-1.